

Pesquisa: plugin-metta-ads (Alisson Oliveira)

O que existe, o que já portamos, e o que ainda falta — documentado pra decisão de produto.

O que existe no plugin-metta-ads do Alisson

Fonte: github.com/Alisson0liveira/plugin-metta-ads (repositório dele, mantenedor Alisson Oliveira — Head de Marketing, Metta). Não faz parte da organização [metta-brasil-tech](#) ; acessei direto pelo GitHub dele.

1. Ele tem modelo de tweet?

Não tem um construído de verdade. Existe uma *categoria teórica* no vocabulário de tratamentos de carrossel dele (`TWEET-CARD` — "statement emulado em card"), mas a própria documentação dele admite:

"sem tweet-card direto no banco; usar a ficha da carta confidencial como família próxima (texto emulado em superfície destacada)"

Ou seja: ele nunca gerou/aprovou um tweet-card de verdade. Quando o sistema dele precisa desse tratamento, usa uma peça diferente (`ad-dark-carta-convite-sistema-pra-poucos-empresarios` — um card estilo "carta confidencial") como aproximação. **Isso bate com o nosso problema:** nós também temos tweet-cards genéricos demais (sem métricas de engajamento, timestamp, logo do X). Nenhum dos dois lados resolveu isso ainda — é um gap real, não só nosso.

2. Ele tem biblioteca de imagens?

Sim — mas não está no repositório do plugin. Vive numa dependência separada, uma skill do Claude Code chamada `metta-brand` , instalada **localmente na máquina dele** em `~/claude/skills/metta-brand/` . Não é um repositório público — procurei (`Alisson0liveira/metta-brand`) e não existe no GitHub. **Não temos acesso a essas imagens reais.**

O que sabemos sobre essa biblioteca (pela documentação, não pelas imagens em si): - **69 fichas** catalogadas em `INDEX-VISUAL.md` , cada uma com 11 campos padronizados: família visual (DARK/LIGHT/YELLOW/COLOR-ACCENT), composição, paleta real (hex), tratamento de foto, headline/sub/CTA literais, presença de marca, elementos extras, tom, vibe. - **≥68 PNGs renderizados** (1080×1920), um por peça — a "fonte da verdade visual" que os agentes dele abrem e olham antes de compor. - Peças do acervo original são SVG (exportado do Figma); peças novas aprovadas entram como HTML. - Regra dele: **"a ficha orienta a busca, mas o PNG é a verdade"** — nunca escolhe referência só pelo texto da ficha, sempre abre a imagem.

Comparação com o nosso banco: nosso `data/applications-index.json` tem **109 peças** (mais que as 69 dele) e já é público/versionado no nosso repo — não depende de instalação local em nenhuma máquina específica. Nesse ponto, estamos à frente.

3. Exemplos de prompts e o caminho até eles

Ele documenta isso muito bem em `lib/metta-image-models.md` — vale a pena adotarmos esse formato. Estrutura de prompt que ele validou:

```
<descrição da cena em inglês> | <iluminação> | <mood> | <color grading> | <ângulo de câmera> | <notas d  
Sempre no final:  
no text, no logos, no signage, photorealistic, cinematic, sharp realism, no hands visible
```

O exemplo real que ele documentou (prompt ruim → bom)

Briefing: anúncio sobre "domingo à noite, peso do WhatsApp não respondido".

❌ **Prompt ruim** (o que ele tentou primeiro):

"Hand holding a smartphone showing WhatsApp messages from team, coffee cup on desk, intimate Sunday evening atmosphere"

Por que falhou: gpt-image-2 não renderiza bolhas de mensagem com texto coerente, e ainda tenta desenhar uma mão — resultado foi "chat vazio + mão estranha".

✅ **Prompt bom** (reformulado, mesma ideia, elementos que o modelo renderiza bem):

"Smartphone face down on a wooden table at dusk, screen glow leaking from underneath, single cup of coffee gone cold beside it, blurred background of empty living room | warm low-key lighting | atmosphere of unanswered burden | desaturated warm tones | close-up at desk level | composition centered on the phone, coffee cup off-frame partial"

O caminho de raciocínio dele (processo formal, dois "papéis" que se checam): 1. **Briefer A (propositor)** define o que a imagem precisa carregar da copy (significado/atmosfera), verifica se dá pra comunicar isso com elementos que o gpt-image-2 renderiza bem, e se não der, **reformula o que a imagem mostra sem mudar o significado** (não força o modelo). 2. **Briefer B (crítico)** valida 3 critérios antes de aprovar: (a) a imagem sozinha comunica a tensão da copy? (b) o gpt-image-2 renderiza isso bem? (c) cabe no template escolhido?

Tabela de onde o gpt-image-2 falha (documentada por ele)

Falha	Sintoma	Como ele contorna
Texto na cena	Letras tortas, bolhas vazias	Pede cena onde texto não importa (tela com gradiente, folha em branco)
Mãos detalhadas	Dedos a mais, mãos retorcidas	Esconde as mãos na composição
Logos de marca	Ícones deformados	Não gera logo — injeta SVG por cima depois
Números específicos	Dígitos trocados/inventados	Nunca embute número na imagem — só via HTML
Rostos reconhecíveis	Rosto genérico mesmo pedindo alguém específico	Mostra de costas, perfil ou sombra
Telas de app (UI)	UI inventada, irreal	Tela apagada/reflexo, ou silhueta do aparelho

Isso é **quase idêntico** ao que já portamos pro nosso `skills/04-image-prompt-engineer.md` (achado #13 do nosso doc de insights) — validação cruzada de que a lista de falhas está certa.

4. Qual é o pipeline dele

Arquitetura fundamentalmente diferente da nossa — vale entender a diferença antes de copiar qualquer coisa.

O dele: orquestração de **subagents do Claude Code** (Task tool), sem código Python de produção — o "pipeline" é uma skill (`SKILL.md`) que o Claude Code interpreta e executa turno a turno, chamando sub-agentes.

O nosso: pipeline **Python determinístico** (LiteLLM + Qdrant + Pillow), roda como função/CLI/serverless, sem depender de uma sessão do Claude Code pra funcionar.

As 9 fases dele (F0–F8)

Fase	O que faz
F0	Recebe o brief, detecta Modo A (copy literal) ou Modo B (ideia → proposta de copy aprovada pelo usuário)
F1 👁	Director abre PNGs de referência do banco (3-5 imagens) e escolhe olhando, não só pela ficha de texto
F1-C 👁	(só carrossel) Direção de série: escolhe 1 de 10 tratamentos <code>T-*</code> , trava paleta P1-P10, define motivos recorrentes
F2	Decide se precisa gerar imagem nova ou não
F3	Dois sub-agentes (Briefer A propositor ↔ Briefer B crítico) refinam o prompt de imagem até 5 rodadas, geram via <code>codex CLI</code>
F4	Spawna designers em paralelo (1 por variante/slide) — cada um compõe HTML olhando a referência
F5	Recebe os resultados: OK, GATE_FAIL ou LAYOUT_IMPOSSIBLE (troca referência se não coube)
F6	Um "crítico visual" compara as peças produzidas lado a lado com as referências do banco
F7 👁	Director abre os PNGs aprovados antes de entregar — sanity check final
F8	Peça aprovada pelo usuário vira entrada nova no banco (ficha + PNG)

👁 = "pontos de visão obrigatórios" — a peça central da filosofia dele: **um agente sempre olha a imagem renderizada**, nunca decide só lendo código/texto.

A lição mais importante dele (por que as versões v0.1–v0.5 falharam)

"O gargalo não era o algoritmo — era a cegueira. O banco exportado do Figma tem texto em `<path>` (SVG) e foto em base64: ilegível como código. O pipeline operava sobre uma representação que não correspondia ao que o olho vê."

A correção dele foi parar de tentar "ler" a peça como código/XML e passar a literalmente abrir o PNG e olhar. É exatamente o motivo de existir o nosso `vision-qa` e o `crítico anti-slop` — resolvemos o mesmo problema, só que via chamada de API de visão (`gpt-4.1` olhando a imagem) em vez de um agente Claude Code usando a ferramenta `Read` .

Gate mecânico dele — o que ele removeu e por quê

Ele tinha regras rígidas (tamanho mínimo de fonte, proporção headline/sub, CTA obrigatoriamente amarelo, marca sempre presente) e **removeu todas** na v0.6 porque o próprio banco de peças campeãs violava essas regras. Ficou só o que é genuinamente binário: copy bate byte a byte, texto na imagem não pode ser inventado, tokens de cor batem com o design system. **Isso é idêntico à lição que já está documentada no nosso `docs/INSIGHTS-PLUGIN-METTA-ADS.md`** — inclusive citamos isso ao portar: "regra mecânica que contradiz o banco é regra errada".

O que já portamos vs. o que ainda não

Já tínhamos investigado isso a fundo antes (docs/INSIGHTS-PLUGIN-METTA-ADS.md , na branch recuperada do backup).
Resumo do que falta:

Do plugin dele	Status no nosso ad-generator
Modos de falha do gpt-image (mãos, texto, logos, números, rostos)	✅ portado — skill 04 v5.0
Vision-QA (olhar o PNG produzido)	✅ temos (_vision_qa.py)
Crítico comparando com referência do banco ("same designer test")	✅ portado (_critic.py) — igual ao dele
Checklist anti-slop	✅ portado (anti-slop.md)
Gate mecânico reduzido (só o binário)	✅ já alinhado
Direção de série (10 tratamentos, paletas P1-P10, regras C1-C8)	⚠️ recuperamos versão nossa do backup, ainda não mergeada
Modo B — ideia bruta → proposta de copy aprovada	⚠️ existe como skill mas pulado no wizard
Tweet-card de verdade	❌ nenhum dos dois lados tem — gap real, precisamos construir do zero
Multi-variante de famílias divergentes por chamada	❌ fora de escopo nos dois (gera 1 peça por vez)

Conclusão prática: não tem "código pra roubar" dele que já não tenhamos processado — o valor real do trabalho dele já foi extraído e documentado. O que resta é decisão de produto (vale implementar Modo B? vale construir direção de série completa?), não descoberta técnica nova.